

Experiment No:03

Name:Anjali Salunkhe

Roll No.:48

Write a C/C++ program for construction of Predictive Parsing Table

Program Code:

```
#include <bits/stdc++.h>
using namespace std;

map<char, vector<string>> grammar;
map<char, set<char>> FIRST, FOLLOW;
set<char> terminals, nonTerminals;
char startSymbol;

// Compute FIRST(X)
set<char> findFirst(string str) {
    set<char> result;
    if (str.empty()) { result.insert('ε'); return result; }

    char first = str[0];
    if (terminals.count(first)) {
        result.insert(first);
        return result;
    }

    for (auto prod : grammar[first]) {
        set<char> temp = findFirst(prod);
        result.insert(temp.begin(), temp.end());
    }
    return result;
}

// Compute FOLLOW
void computeFollow() {
    FOLLOW[startSymbol].insert('$');
    bool changed = true;
    while (changed) {
        changed = false;
        for (auto &rule : grammar) {
            char A = rule.first;
            for (string prod : rule.second) {
                for (int i = 0; i < prod.size(); i++) {
                    char B = prod[i];
                    if (nonTerminals.count(B)) {
                        set<char> firstBeta;
                        if (i+1 < prod.size()) {
                            firstBeta = findFirst(prod.substr(i+1));
                        } else {
                            firstBeta.insert('ε');
                        }
                        int oldSize = FOLLOW[B].size();
                        for (char f : firstBeta) {
                            if (f != 'ε') FOLLOW[B].insert(f);
                        }
                        if (firstBeta.count('ε') || i+1 == prod.size()) {
                            FOLLOW[B].insert(FOLLOW[A].begin(), FOLLOW[A].end());
                        }
                        if (FOLLOW[B].size() > oldSize) changed = true;
                    }
                }
            }
        }
    }
}
```

```

int main() {
    int n;
    cout << "Enter number of grammar productions: ";
    cin >> n;
    cout << "Enter productions (Format: A->XYZ), use 'ε' for epsilon:\n";

    for (int i = 0; i < n; i++) {
        string prod;
        cin >> prod;
        char lhs = prod[0];
        string rhs = prod.substr(3);
        grammar[lhs].push_back(rhs);
        nonTerminals.insert(lhs);
        for (char c : rhs) {
            if (!isupper(c) && c != 'ε') terminals.insert(c);
        }
    }

    startSymbol = grammar.begin()->first;

    // Compute FIRST sets
    for (auto &nt : nonTerminals) {
        for (auto &prod : grammar[nt]) {
            set<char> f = findFirst(prod);
            FIRST[nt].insert(f.begin(), f.end());
        }
    }

    // Compute FOLLOW sets
    computeFollow();

    cout << "\nFIRST Sets:\n";
    for (auto &p : FIRST) {
        cout << "FIRST(" << p.first << ") = { ";
        for (char c : p.second) cout << c << " ";
        cout << "}\n";
    }

    cout << "\nFOLLOW Sets:\n";
    for (auto &p : FOLLOW) {
        cout << "FOLLOW(" << p.first << ") = { ";
        for (char c : p.second) cout << c << " ";
        cout << "}\n";
    }

    // Construct Predictive Parsing Table
    cout << "\nPredictive Parsing Table:\n";
    vector<char> termList(terminals.begin(), terminals.end());
    termList.push_back('$');

    cout << "\t";
    for (char t : termList) cout << t << "\t";
    cout << "\n";

    for (char nt : nonTerminals) {
        cout << nt << "\t";
        for (char t : termList) {
            string entry = "";
            for (string prod : grammar[nt]) {
                set<char> f = findFirst(prod);
                if (f.count(t)) entry = prod;
                if (f.count('ε') && FOLLOW[nt].count(t)) entry = prod;
            }
            if (entry != "") cout << nt << "->" << entry << "\t";
            else cout << "-\t";
        }
        cout << "\n";
    }
}

```

```

        cout << "\nResult : Program for construction of Predictive Parsing Table was successfully executed "
        return 0;
}

```

Actual Output:

Enter number of grammar productions: 5

E->TE'

E'->+TE'

E'-> ϵ

T->FT'

T'-> ϵ

FIRST Sets:

FIRST(E) = { (id }

FIRST(E') = { + ϵ }

FIRST(T) = { (id }

FIRST(T') = { ϵ }

FIRST(F) = { (id }

FOLLOW Sets:

FOLLOW(E) = { \$) }

FOLLOW(E') = { \$) }

FOLLOW(T) = { + \$) }

FOLLOW(T') = { + \$) }

FOLLOW(F) = { * + \$) }

Predictive Parsing Table:

	id	(	+	\$
E	E->TE'	E->TE'	-	-
E'	-	-	E'-> ϵ	E'->+TE' E'-> ϵ
T	T->FT'	T->FT'	-	-
T'	-	-	T'-> ϵ	T'-> ϵ
F	F->id	F->(E)	-	-

Result : Program for construction of Predictive Parsing Table was successfully executed